UNIMINUTO
Corporación Universitaria Minuto de Dios
Educación de calidad al alcance de todos
Vigilada MinEducación
ALTA CALIDAD

Desarrollo de proyecto en Scratch con GitHub

Salomé Murcia Jiménez

001140148

Ingeniería de Sistemas, Facultad de Ingeniería, Corporación Universitaria Minuto de Dios

NRC 82336: Programación Básica

Mg. Jasson Diaz Zamudio

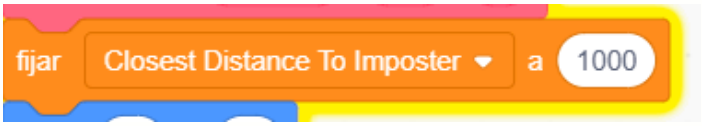
Marzo 16, 2026

Informe del proyecto Scratch

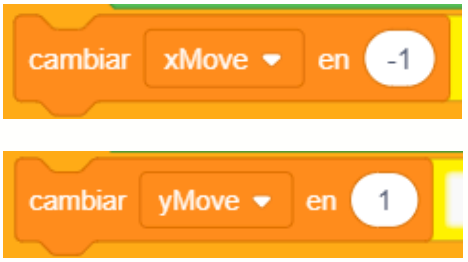
- **¿Qué variables utilizaste en tu proyecto y cuál es la función de cada una dentro del juego? Incluye el bloque donde se crea o modifica una variable.**

  - **Kill Cooldown:** es el tiempo que hay que esperar para volver a atacar. Funciona como temporizador de espera. Su objetivo es que el juego sea justo porque evita que el impostor pueda eliminar a todos los tripulantes seguidos. Cuando comienza el juego, la variable se establece en un valor inicial. Luego el valor disminuye hasta llegar a 0, momento en el que el jugador puede volver a eliminar a otro tripulante. Si el número no está en 0, la acción de matar está bloqueada.

  - **Closest Distance To Imposter y Closest Crewmate Index:** Ambas variables se usaron para manejar la lógica de proximidad. La primera mide constantemente qué tan cerca está el tripulante más próximo, si esta distancia es corta, o sea menor al rango definido, la variable avisa al sistema que se puede atacar. Y la segunda identifica específicamente a cuál de los tripulantes se debe eliminar basándose en su posición en las listas de coordenadas.



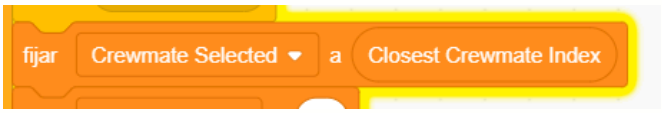
- **xMove / yMove:** funcionan como indicadores de velocidad y dirección dentro del plano del juego. En lugar de mover al personaje instantáneamente, el programa registra primero la intención del jugador: xMove gestiona el desplazamiento lateral (derecha e izquierda) mientras que yMove se encarga del eje vertical (arriba y abajo). Cuando se presiona una tecla, la variable guarda un valor (1 para positivo o -1 para negativo) que determina hacia donde debe apuntar el movimiento.



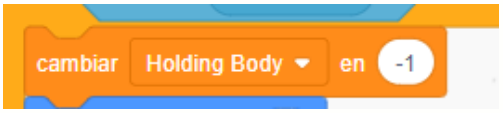
- **Crewmates Alive:** Esta variable actúa como un contador dinámico de población. Al inicio, suma +1 por cada tripulante creado para registrar el total de objetivos. Durante la partida, cada vez que un tripulante es eliminado, el valor disminuye en -1. Cuando la variable llega a 0, el sistema detecta que ya no quedan objetivos y activa automáticamente el evento de victoria (Game Won).

-

- **Crewmate selected**: su función es señalar e identificar qué tripulante está siendo seleccionado por el impostor en el momento de realizar la eliminación. Esta variable permite que el juego determine cuál personaje debe ser eliminado cuando el jugador presione la tecla de ataque.



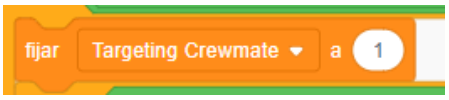
- **Holding Body:** se utiliza para indicar si el jugador está cargando un cuerpo dentro del juego. Cuando se presiona la bandera verde, la variable se fija en 0, lo que significa que el jugador no está cargando ningún cuerpo al inicio de la partida.



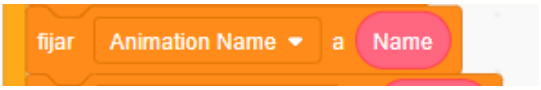
- **Kill button pressed:** su función es detectar cuándo el jugador presiona el botón de eliminar. El jugador al presionar la tecla espacio, el valor de la variable cambia para indicar que la acción de eliminación fue activada.

- **Targeting Crewmate**: se utiliza para detectar si el impostor está apuntando o se encuentra cerca de un tripulante que puede ser eliminado. Esta variable funciona con dos estados: cuando su valor es **0** significa que el

impostor no está apuntando a ningún tripulante, y cuando su valor es **1** significa que el impostor está apuntando a un tripulante y puede realizar la eliminación. Esto permite que el juego solo active la acción de eliminar cuando el impostor está lo suficientemente cerca de un tripulante.

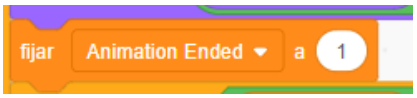


- **Animation Name:** guarda el nombre de la animación que se está reproduciendo en ese momento en el personaje. Esto permite que el programa sepa qué animación debe mostrar, por ejemplo, cuando el personaje camina, se detiene o realiza otra acción dentro del juego.



- **Animation Wait Time:** controla el tiempo de espera entre los cambios de una animación. Esto permite que los cuadros de la animación se reproduzcan a una velocidad adecuada y que el movimiento del personaje se vea más natural.

- **Animation Ended**: indicar si una animación ha terminado. Cuando el valor es **0**, significa que la animación aún se está reproduciendo, y cuando el valor es **1**, significa que la animación ya terminó.

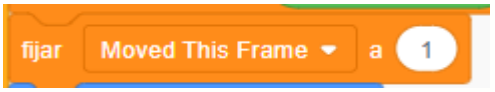


- **Animation Looping:** se utiliza para indicar si una animación debe repetirse continuamente. Cuando el valor es **1**, la animación se repite en bucle, y cuando el valor es **0**, la animación se reproduce solo una vez.

- **TempDir/ tempVariable:** se utiliza como una variable temporal para guardar información momentánea durante la ejecución del programa, como una dirección o un valor que se necesita para realizar cálculos o movimientos dentro del juego.

-

- **Moved This Frame:** se utiliza para detectar si el personaje ya se movió durante ese momento del juego. Esto ayuda a evitar que se ejecuten varios movimientos al mismo tiempo, por ejemplo, cuando el jugador presiona más de una tecla. Cuando la variable tiene el valor **0**, significa que el personaje aún no se ha movido en ese momento, y cuando tiene el valor **1**, indica que el personaje ya realizó un movimiento, evitando que se mezclen varios movimientos en el mismo instante.



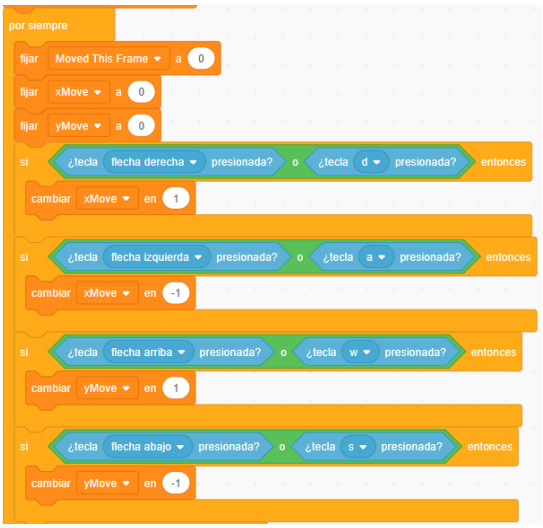
- **Trampa activada**: su función es actuar como interruptor de estado que indica si el impostor ha caído en la trampa. Al presionar la bandera verde, la variable se inicializa en 0, lo que permite que el impostor se desplace libremente por el escenario. Si el sensor de contacto detecta que el personaje toca el objeto Sprite 1 (el rombo), la variable cambia instantáneamente a 1, esto hace que el programa dispare la secuencia de derrota, enviando el mensaje de “muerte” para ocultar al personaje y detener su movimiento.

- **Explica cómo una variable afecta el comportamiento del personaje o del juego. Copia el fragmento de bloques donde se utilice esa variable**.

La variable xMove se utiliza para controlar el movimiento horizontal del personaje (impostor) dentro del juego. Esta variable guarda el valor que indica cuánto debe moverse el personaje hacia la izquierda o hacia la derecha.

Esta variable afecta directamente el comportamiento del personaje, ya que determina si el impostor se mueve y en qué dirección lo hace cuando el jugador presiona las teclas de movimiento. Por ejemplo, cuando se presiona la tecla para moverse a la derecha, el valor de xMove permite que el personaje avance hacia ese lado. De la misma forma, cuando se mueve hacia la izquierda, el valor de la variable cambia para mover al personaje en la dirección contraria.

De esta manera, la variable influye en la forma en que el jugador controla al impostor y cómo se desplaza por el escenario, lo cual afecta directamente el desarrollo del juego.



- **Identifica una estructura si (if) utilizada en tu programa. Explica qué condición evalúa y qué sucede cuando la condición se cumple o no se cumple. Incluye el bloque correspondiente.**

Identifiqué la estructura condicional Si (if) dentro del bloque personalizado de muerte.

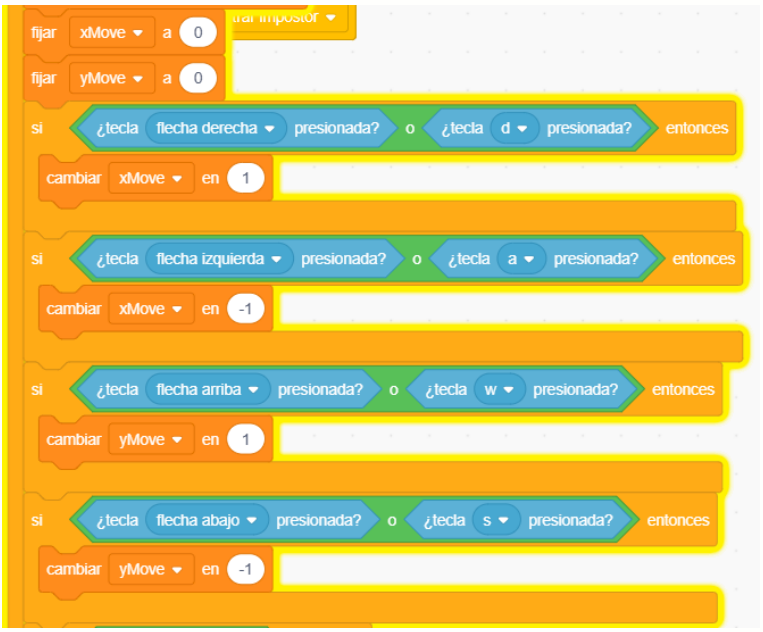
Evalúa si el sensor de contacto detecta que el impostor está tocando el objeto Sprite 1 (la trampa, o sea el rombo)

Cuando la condición si se cumple, el programa cambia la variable trampa activada a 1 y envía el mensaje “muerte” para que el personaje desaparezca. Por el contrario, si no se cumple, el programa ignora las instrucciones internas y el impostor puede seguir caminando normalmente por esa zona.

- **Describe cómo utilizaste estructuras condicionales para controlar el movimiento del personaje en diferentes direcciones. Copia el bloque donde se detecta la tecla presionada.**

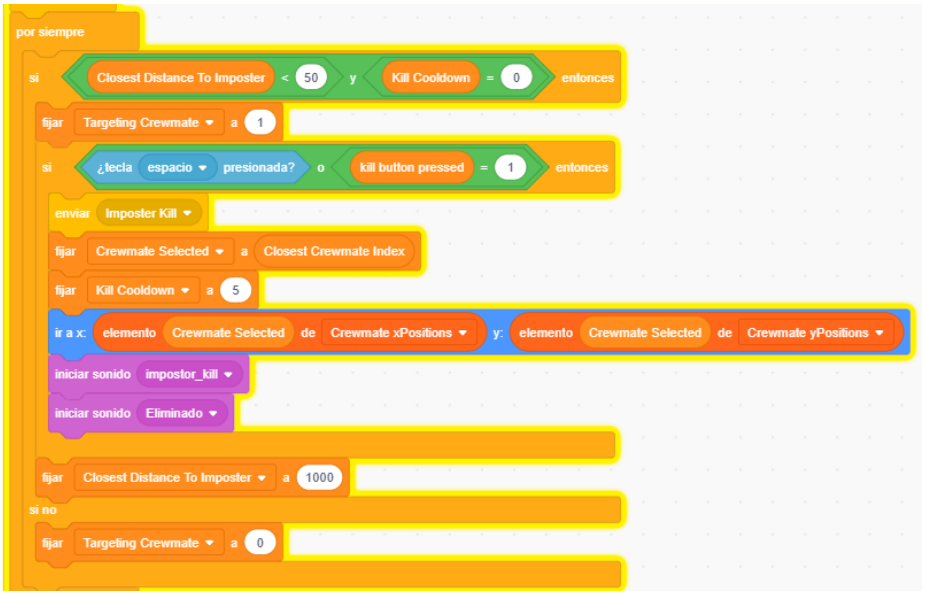
Las variables xMove y yMove se utilizan para controlar el movimiento del personaje (impostor) en los ejes horizontal y vertical dentro del escenario. Estas variables funcionan

como un registro de dirección que indica hacia dónde debe desplazarse el personaje en cada momento. xMove controla el movimiento hacia la izquierda o hacia la derecha, mientras que yMove controla el movimiento hacia arriba o hacia abajo, lo que afecta directamente el comportamiento del impostor dentro del juego. Para mejorar la jugabilidad, el programa detecta constantemente la entrada del teclado utilizando tanto las flechas de dirección como las teclas W, A, S y D. Cuando el jugador presiona, por ejemplo, la tecla D o la flecha derecha, la variable xMove cambia su valor a 1, indicando un movimiento positivo en el eje X. De manera similar, al presionar W o la flecha arriba, la variable yMove cambia a 1 para mover al personaje hacia arriba. Cuando las teclas se dejan de presionar, ambas variables regresan a 0, lo que detiene el movimiento del personaje. Gracias a esto, el jugador puede incluso moverse en diagonal al presionar dos teclas al mismo tiempo, lo que permite un control más fluido y preciso del impostor durante el juego.



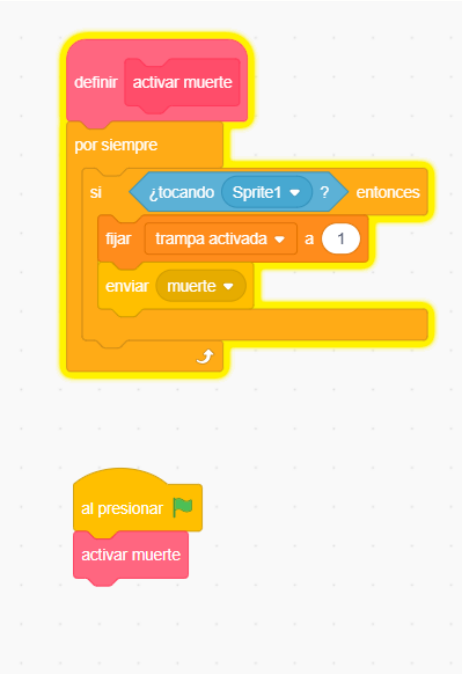
- **Explica un ejemplo dentro del juego donde una condición permite tomar una decisión (por ejemplo colisiones, puntaje, cambio de estado, etc.).**

En este proyecto, se implementó un sistema de toma de decisiones basado en condiciones múltiples para gestionar la mecánica de ataque del impostor. El programa evalúa constantemente si el jugador ha presionado la tecla espacio mientras se cumplen dos requisitos técnicos adicionales: que la distancia respecto al tripulante sea menor a 50 unidades y que la variable Kill Cooldown haya llegado a cero. Si todas estas condiciones son verdaderas, el sistema decide ejecutar un cambio de estado en el juego, disparando el evento Imposter Kill que activa la animación de eliminación y los efectos de sonido correspondientes. Por el contrario, si alguna de las condiciones falla (por ejemplo, si el ataque aún está en tiempo de recarga), el programa decide ignorar la entrada del usuario, manteniendo la integridad de las reglas del juego y evitando acciones no permitidas en ese momento.



- **Indica si creaste bloques personalizados. Si lo hiciste, explica cuál es su función y muestra el bloque creado.**

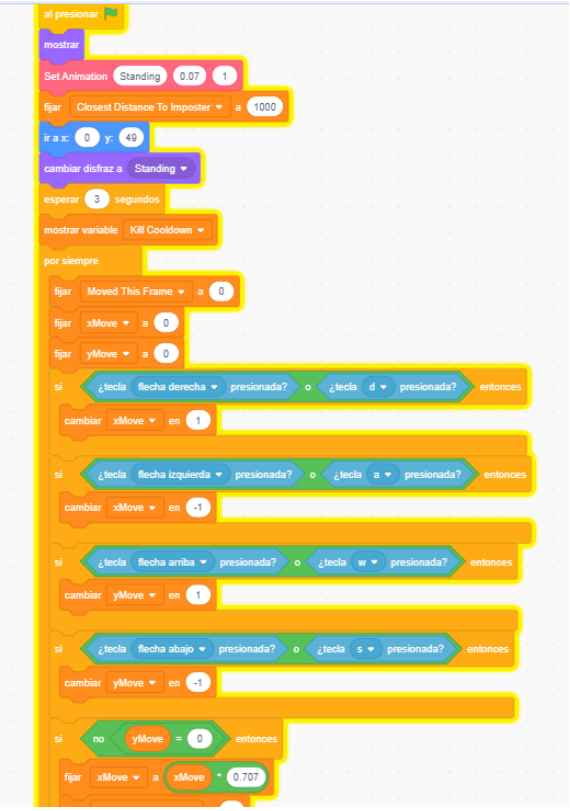
Si, para este proyecto cree un bloque personalizado llamado “activar muerte”. Lo hice principalmente para que mi código estuviera más ordenado y no tener tantos bloques amontonados bajo la bandera verde. Este bloque se encarga de estar revisando todo el tiempo si el impostor toca el rombo (Sprite1) en el momento en que lo toca, el bloque hace que la variable trampa activada cambie a 1 y manda el aviso de “muerte” para que el personaje se esconda. Básicamente, lo cree para que toda esa lógica de la colisión quedara guardada en un solo lugar y el programa fuera más fácil de entender

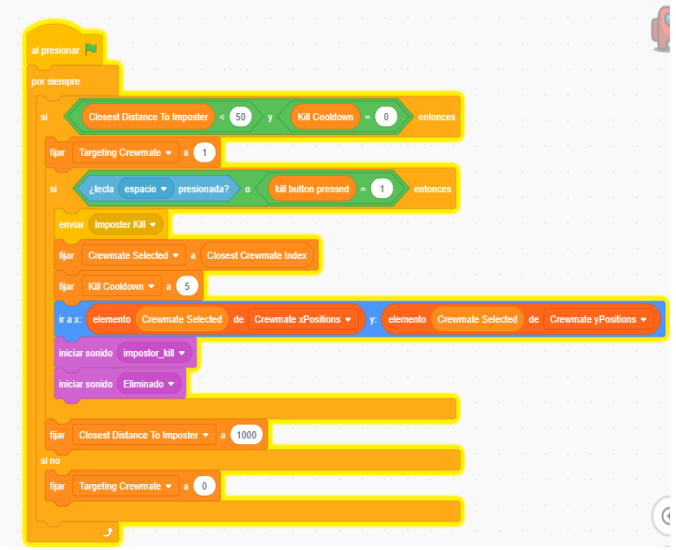


- **En el ejercicio de replicación del juego, explica qué parte del código original fue más importante para comprender su funcionamiento. Copia el fragmento equivalente de tu proyecto.**

Considero que la parte más importante para entender el funcionamiento del juego fue comprender la interacción entre las entradas del teclado y las variables que controlan las acciones del personaje. Fue fundamental entender cómo, al presionar una tecla de movimiento o la barra de espacio, el programa procesa esa orden para que el personaje se desplace o ejecute un ataque. Esto permitió observar que la programación funciona mediante condicionales, ya que el sistema evalúa si el jugador realiza una acción y, dependiendo de ello, ejecuta una respuesta.

Además, fue clave comprender cómo las variables controlan las reglas del juego. Por ejemplo, utilicé variables para determinar si una trampa ha sido activada o para gestionar el tiempo de espera necesario entre cada ataque. Esto demuestra que un programa funciona como un conjunto de datos que cambian constantemente según las decisiones del usuario y las condiciones del entorno, permitiendo que el sistema responda de manera lógica a cada situación.





- **Describe la innovación que agregaste al juego replicado y explica qué bloques utilizaste para implementarla.**

“3, 2, 1, ¡GO!”

Una de las innovaciones que agregué al juego fue una pantalla de inicio con una cuenta regresiva de “3, 2, 1, ¡GO!” antes de que comience la partida. Esta animación sirve para preparar al jugador antes de iniciar el juego y hace que el inicio sea más dinámico y claro.

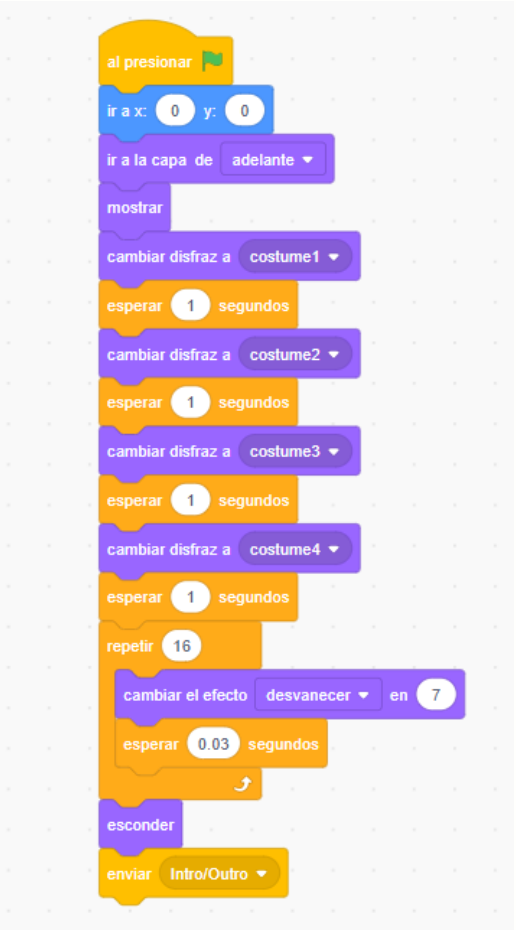
Para implementar esta función utilicé varios bloques de Scratch que controlan la posición, la apariencia y el tiempo de la animación.

Primero utilicé el bloque “al presionar bandera verde”, que permite que la animación comience automáticamente cuando se inicia el juego. Luego utilicé “ir a x: 0 y: 0” para colocar el sprite de la cuenta regresiva en el centro del escenario, también “ir a la capa de adelante” para asegurar que el mensaje aparezca por encima de los demás objetos.

Después usé el bloque “mostrar” para que el sprite sea visible y “cambiar disfraz” para mostrar cada número de la cuenta regresiva. Cada disfraz representa un momento diferente de la animación: primero el número 3, luego 2, después 1, y finalmente GO. Entre cada cambio de disfraz utilicé el bloque “esperar 1 segundo” para que cada número se muestre durante un momento antes de pasar al siguiente.

Una vez que aparece el mensaje final, utilicé el bloque “repetir 16 veces” junto con “cambiar efecto desvanecer por 7” para crear un efecto visual en el que el texto desaparece poco a poco. El número 16 se utiliza para aplicar el efecto varias veces seguidas y lograr que la desaparición del mensaje sea gradual y suave, en lugar de desaparecer de forma instantánea. Dentro de ese ciclo también se utilizó “esperar 0.03 segundos” para que cada cambio del efecto ocurra con un pequeño intervalo de tiempo, haciendo que la animación se vea más fluida.

Finalmente utilicé el bloque “esconder” para que el sprite desaparezca completamente y “enviar mensaje intro/outro” para indicar que la animación terminó y permitir que el juego continúe con la siguiente parte.



Aviso visual y sonoro de “Eliminado”

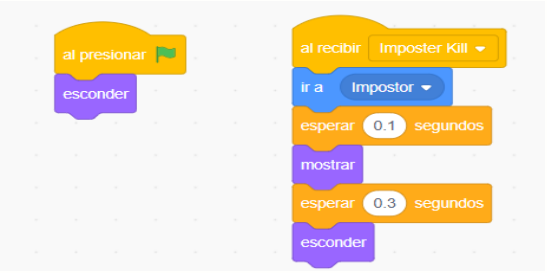
La segunda innovación que agregué al juego fue un aviso visual y sonoro cuando el impostor elimina a un tripulante. Cuando el jugador realiza una eliminación, aparece un letrero que dice “Eliminado” y al mismo tiempo se reproduce un sonido que confirma la

acción. Esta mejora permite que el jugador tenga una confirmación clara de que el tripulante ha sido eliminado.

Para implementar esta función primero creé un nuevo sprite que contiene el letrero con la palabra “Eliminado”. En este objeto utilicé varios bloques para controlar cuándo debe aparecer y desaparecer el mensaje.

El bloque “al presionar bandera verde → esconder” se utiliza para que el letrero esté oculto cuando el juego comienza. Luego se utilizó el bloque “al recibir imposter kill”, que es un mensaje que se envía cuando el impostor elimina a un tripulante. Cuando el sprite recibe este mensaje, ejecuta varias acciones: primero utiliza “ir a impostor” para colocarse cerca del personaje que realizó la eliminación, luego espera 0.1 segundos, después utiliza el bloque “mostrar” para que el mensaje aparezca en pantalla, y finalmente espera 0.3 segundos y se esconde para que el aviso desaparezca rápidamente.

Además del aviso visual, también agregué un efecto de sonido para reforzar la acción. En el código donde ya estaba programada la eliminación del tripulante (cuando el impostor está cerca y se presiona la tecla espacio), agregué el bloque “iniciar sonido ‘Eliminado’”. De esta forma, cada vez que ocurre una eliminación, el juego reproduce el sonido al mismo tiempo que aparece el letrero.



Trampas al impostor

La tercera innovación que agregué al juego fue un sistema de trampas que puede eliminar al impostor si este pasa por ciertas zonas del escenario. Para esto creé un nuevo objeto que contiene cuatro rombos, los cuales representan las trampas colocadas en el suelo del mapa.

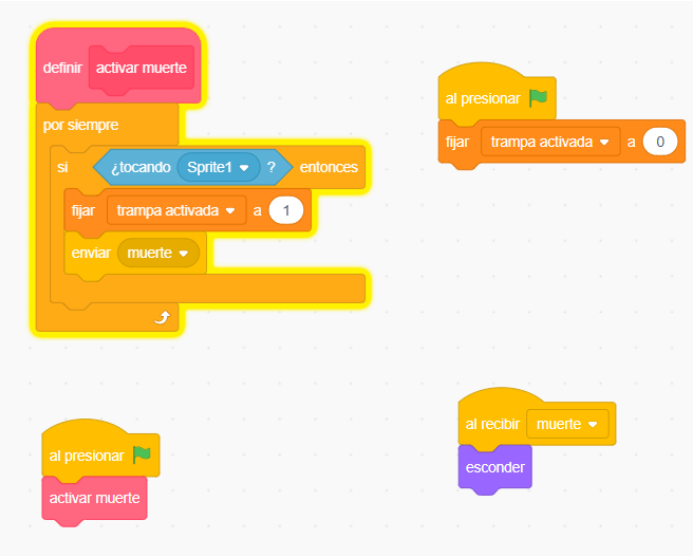
En el sprite de las trampas utilicé varios bloques para controlar cuándo deben aparecer o desaparecer. El bloque “al presionar bandera verde → esperar 1 segundo → mostrar” se utilizó para que las trampas aparezcan en el escenario después de iniciar el juego.

También agregué el bloque “al recibir muerte → esconder”, que hace que las trampas desaparezcan cuando ocurre la eliminación del impostor.

Para detectar cuándo el impostor pisa una trampa, agregué código en el sprite del impostor. Allí creé un bloque personalizado llamado “activar muerte”, que se encarga de verificar constantemente si el personaje está tocando una trampa. Dentro de este bloque utilicé un bucle “por siempre” junto con una condición “si tocando sprite (rombos)”.

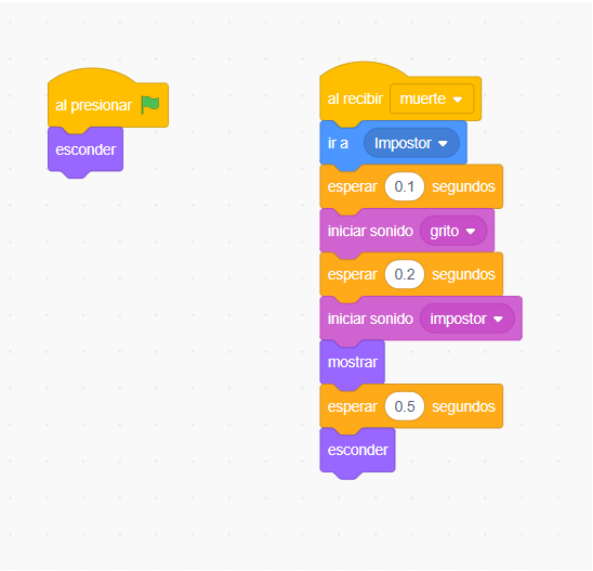
Cuando esta condición se cumple, el programa fija la variable “trampa activada” a 1 y envía el mensaje “muerte”, lo que provoca que el impostor sea eliminado.

También, al iniciar el juego utilicé el bloque “al presionar bandera verde → fijar trampa activada a 0” para reiniciar el estado de la variable y asegurar que el sistema de trampas funcione correctamente desde el comienzo.



Además de eliminar al personaje, agregué un aviso visual y sonoro para que el jugador entienda claramente lo que ocurrió. Para esto utilicé un sprite con un letrero que dice “Eliminado” que aparece cuando el impostor cae en la trampa. En este sprite utilicé los siguientes bloques: primero “al presionar bandera verde → esconder” para que el letrero no sea visible al inicio del juego. Luego utilicé “al recibir muerte”, que se activa cuando el impostor toca una trampa. En ese momento el sprite ejecuta varias acciones: “ir a impostor” para colocarse cerca del personaje, esperar 0.1 segundos, luego iniciar sonido

“grito” para simular que el impostor grita al caer en la trampa. Después espera 0.2 segundos y ejecuta “iniciar sonido ‘impostor eliminado’”, indicando que el personaje ha sido eliminado. Al mismo tiempo el letrero se muestra en pantalla, permanece visible 0.5 segundos, y finalmente se esconde nuevamente.



- **Explica una dificultad que surgió durante el desarrollo del proyecto y cómo la solucionaste.**

La mayor dificultad que tuve fue entender qué eran las variables y cómo se utilizaban en el programa. Al principio no comprendía cómo funcionaban, y muchas veces colocaba códigos, pero no obtenía el resultado que esperaba.

Para solucionarlo, me dediqué a indagar, investigar y experimentar con los bloques, probando diferentes combinaciones hasta que funcionara como quería. Poco a poco fui comprendiendo cómo cada bloque y cada variable afectaba al juego.

- **Describe qué aprendiste sobre el uso de variables, condicionales y bloques durante el desarrollo de esta actividad.**

Durante esta actividad aprendí que las variables sirven para guardar información que cambia mientras se juega, como el estado de los personajes o el tiempo de espera entre acciones. Comprender cómo funcionan permitió que los personajes respondieran correctamente a las acciones que el jugador les asigna.

Las condicionales (si … entonces) me ayudaron a que el juego tome decisiones dependiendo de lo que pasa, por ejemplo, ejecutar una acción solo cuando se cumple cierta condición. Gracias a ellas entendí que sin estas estructuras el juego no podría reaccionar de manera lógica a lo que hace el jugador.

Por último, los bloques me enseñaron a organizar las acciones del juego de manera clara y ordenada. Al combinarlos con las variables y condicionales, pude lograr que los personajes se movieran, cambiaran de disfraz, aparecieran mensajes y sonidos, y en general que todo funcione correctamente. Esta actividad me permitió comprender cómo estas tres herramientas trabajan juntas y lo importante que es combinarlas bien para que el juego funcione como uno espera.